

Git Lab — Built Around a Real Project (StudentGradeApp)

Instead of empty text files, every experiment below adds a real, working piece to the same small Python project: **StudentGradeApp**, a command-line tool that manages student names and marks. By Experiment 12, students have a genuinely functioning app — and every Git concept was learned *while building something*, not on throwaway files.

No external libraries needed — pure Python 3, which Ubuntu ships with by default (`python3 --version` to confirm). This keeps the lab focused on Git, not on environment setup.

The project's evolution, at a glance

Experiment	What gets added to the app
1	<code>app.py</code> with <code>add_student()</code>
2	<code>calculate_average()</code> — built on feature-branch, merged in
3	Mid-write on <code>grade_letter()</code> — interrupted, stashed, resumed
4	Cloning the project onto a "second machine"
5	Pulling a teammate's remote update via fetch + rebase
6	Finishing and merging <code>grade_letter()</code> with a custom message
7	Tagging the first working version <code>v1.0</code>
8	Cherry-picking <code>save_to_file()</code> / <code>load_from_file()</code> from a side branch
9–11	Inspecting the project's own commit history
12	Reverting a commit that broke <code>calculate_average()</code>
Bonus A	A real conflict — two edits to <code>grade_letter()</code> at once
Bonus B	<code>.gitignore</code> for <code>__pycache__</code> , generated data files
Bonus C	A peer contributes <code>top_scorer()</code> via fork + pull request

Experiment 1 — Initialize repo, add the first real function



```
bash
mkdir StudentGradeApp
cd StudentGradeApp
git init
```

Create `app.py`:



```
bash
nano app.py
```



```
python
students = []

def add_student(name, marks):
    students.append({"name": name, "marks": marks})
    print(f"Added {name} with marks {marks}")

if __name__ == "__main__":
    add_student("Asha", 88)
    print(students)
```



```
bash
git add app.py
git status
git commit -m "Initial commit: added add_student function"
python3 app.py          # run it — students should see real output
```

Experiment 2 — Branch + merge: add `calculate_average()`



```
bash
cd StudentGradeApp
git branch feature-branch
git checkout feature-branch
```

Edit `app.py`, add this function below `add_student`:



```
python
def calculate_average(students):
    total = sum(s["marks"] for s in students)
    return total / len(students) if students else 0
```

Also update the demo block at the bottom to call it:



```
python
if __name__ == "__main__":
    add_student("Asha", 88)
    add_student("Ravi", 76)
    print("Average:", calculate_average(students))
```



```
bash
git add app.py
git commit -m "Added calculate_average function"

git checkout master
git merge feature-branch
python3 app.py          # confirm the merged version runs correctly
```

Experiment 3 — Stash mid-feature, switch, resume

Start writing `grade_letter()` but get interrupted before finishing:



```
bash
nano app.py
```

Add (deliberately incomplete):



```
python
def grade_letter(marks):
    if marks >= 90:
        return "A"
    # TODO: finish remaining grade boundaries
```



```
bash
git stash          # shelve the incomplete function
git status         # confirm working directory is clean

git checkout feature-branch # go check something on the other branch
git checkout master      # come back

git stash apply      # bring the incomplete function back
```

Leave it incomplete for now — it gets finished and merged properly in Experiment 6.

Experiment 4 — Clone the project (simulating a new machine / new teammate)

First, push what exists so far to GitHub (one-time, if not already done):

```

bash
git remote add origin https://github.com/<username>/StudentGradeApp.git
git push -u origin master
```

Now simulate a fresh checkout elsewhere:

```

bash
cd ~
git clone https://github.com/<username>/StudentGradeApp.git StudentGradeApp-clone
cd StudentGradeApp-clone
python3 app.py          # confirm it runs identically to the original
```

Experiment 5 — Fetch + rebase a teammate's update

Simulate a teammate's change: edit `README.md` (or add one) directly on GitHub's website — e.g. add a line describing the project — then commit it there. Now pull it into the clone:

```

bash
cd StudentGradeApp-clone
git fetch origin
git rebase origin/master
```

If working solo, simulate the "teammate" by making a small commit directly in the *original* `StudentGradeApp` folder and pushing it, then rebasing in the clone.

Experiment 6 — Finish `grade_letter()`, merge with a custom message

Back in the original repo, finish the function properly:

```

bash
cd StudentGradeApp
nano app.py
```

```

python
def grade_letter(marks):
    if marks >= 90:
        return "A"
    elif marks >= 75:
        return "B"
    elif marks >= 60:
        return "C"
    else:
        return "D"
```

```

bash
git add app.py
git commit -m "Complete grade_letter function"
git checkout master
git merge feature-branch -m "Merged feature-branch into master: completed grade_letter function"
```

Experiment 7 — Tag the first working release

At this point the app has `add_student`, `calculate_average`, and `grade_letter` — a genuinely usable first version:

```

bash
git tag v1.0
git tag          # confirm it's listed
```

Experiment 8 — Cherry-pick persistence functions from a side branch



```
bash
git checkout -b source-branch

nano app.py
```

Add:



```
python
def save_to_file(students, filename="students_data.txt"):
    with open(filename, "w") as f:
        for s in students:
            f.write(f"{s['name']},{s['marks']}\n")
```



```
bash
git add app.py
git commit -m "Add save_to_file function"

nano app.py
```

Add:



```
python
def load_from_file(filename="students_data.txt"):
    loaded = []
    with open(filename) as f:
        for line in f:
            name, marks = line.strip().split(",")
            loaded.append({"name": name, "marks": int(marks)})
    return loaded
```



```
bash
```

```
git add app.py
git commit -m "Add load_from_file function"

git log --oneline          # note the two commit hashes

git checkout master
git cherry-pick <save_commit_hash>^.<load_commit_hash>
python3 app.py            # confirm master now has persistence too
```

Experiment 9 — Inspect a specific commit

```

bash
git log --oneline          # find the commit that added calculate_average
git show <that-commit-id> # see full diff, author, date, message
```

Experiment 10 — Commits by author within a date range

```

bash
git log --author="<student-name>" --since="2025-01-01" --until="2025-12-31"
```

Experiment 11 — Last five commits

```

bash
git log -5
git log -5 --oneline
```

Experiment 12 — Revert a commit that broke the app

Simulate a bug: introduce a divide-by-zero risk into `calculate_average`:

```

bash
nano app.py
```

Change:



```
python
def calculate_average(students):
    total = sum(s["marks"] for s in students)
    return total / len(students)    # bug: crashes when students is empty
```



```
bash
git add app.py
git commit -m "Buggy change: removed empty-list check in calculate_average"

python3 -c "from app import calculate_average; print(calculate_average([]))"
# this crashes with ZeroDivisionError — the "bug" is demonstrated live

git log --oneline          # find the bad commit's ID
git revert <bad-commit-id>
python3 -c "from app import calculate_average; print(calculate_average([]))"
# now safely returns 0 again
```

Bonus A — A real merge conflict in `grade_letter()`



```
bash
git checkout master
nano app.py
```

Change the grade boundaries directly on master:



```
python
def grade_letter(marks):
    if marks >= 85:    # changed from 90
        return "A"
    ...
```



```
bash
```

```
git add app.py
git commit -m "Tighten A-grade boundary on master"

git checkout -b grading-policy-branch
nano app.py
```

Change the *same* line differently:

```
☐

python
def grade_letter(marks):
    if marks >= 95:      # changed from 90, differently
        return "A"
    ...
```

```
☐

bash
git add app.py
git commit -m "Tighten A-grade boundary on grading-policy-branch"

git checkout master
git merge grading-policy-branch    # CONFLICT — both changed the same line
```

Inspect and resolve exactly as covered earlier in the lab: open `app.py`, find the `<<<<<< / ===== / >>>>>>` markers around the `grade_letter` function, decide on one boundary (or a compromise), remove the markers, then:

```
☐

bash
git add app.py
git commit -m "Resolved grading boundary conflict — settled on 90"
```

Bonus B — `.gitignore` for real generated files

By Experiment 8, running the app creates `students_data.txt`, and Python creates a `__pycache__ /` folder the moment any module is imported. Both should never be tracked:

```
☐

bash
git status    # __pycache__ / and students_data.txt now show as untracked
```

```
☐

bash
```

`nano .gitignore`

```

# Python bytecode cache
__pycache__/*
*.pyc

# runtime-generated student data, not source code
students_data.txt
```

```

bash
git status          # both are now correctly ignored
git add .gitignore
git commit -m "Add .gitignore for cache and generated data files"
```

Bonus C — Peer contributes `top_scorer()` via fork + pull request

Pair up two students (A = owner, B = contributor) as before, but the contribution is now a real feature:

(*GitHub website*) Student B forks Student A's `StudentGradeApp`.

Student B clones their fork, branches, and adds:

```

python
def top_scorer(students):
    return max(students, key=lambda s: s["marks"]) if students else None
```

```

bash
git checkout -b add-top-scorer
git add app.py
git commit -m "Add top_scorer function"
git push origin add-top-scorer
```

(*GitHub website*) Student B opens a pull request against Student A's repo.

(*GitHub website*) Student A reviews the diff — checks that `top_scorer` handles the empty-list case sensibly (it does, returns `None`) — and merges.

Student B syncs their fork:

```

bash
```

```
git checkout master
git remote add upstream https://github.com/<student-A-username>/StudentGradeApp.git
git fetch upstream
git merge upstream/master
```

Why this approach is worth the extra setup

Students finish the lab with:

A working multi-function Python app they built commit-by-commit

A commit history that tells a real story (`git log --oneline --graph --all`) looks meaningful, not like 12 disconnected test commits)

First-hand experience of *why* each Git feature exists — stash because they got interrupted mid-function, revert because a real change introduced a real bug, conflict because two people genuinely touched the same logic, .gitignore because the app genuinely generates files nobody wants tracked

This also makes lab records more meaningful to grade against the rubric's "Design and Implementation" and "Result & Analysis" rows — there's an actual working result to analyze, not just command output.